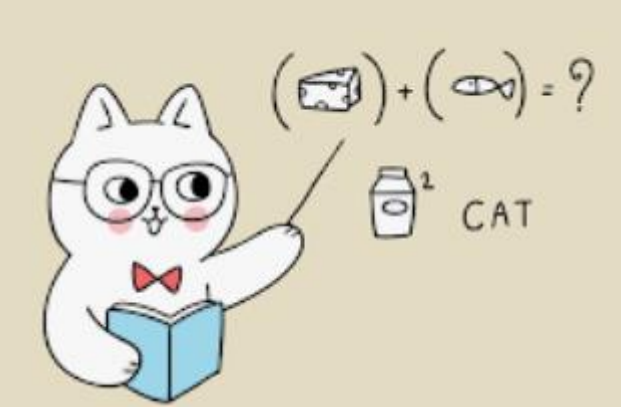
ST443 Week 1

Getting started

1 October 2026

These slides are not official notes. Always refer to the official material on Moodle

Phil Chan



- Admin

- Guide to lab 1:
  - NumPy
  - Matplotlib
  - pandas
- Linear algebra review



Dr Phil Chan

Guest Teacher

Department of Statistics

Email p.chan@lse.ac.uk

Room

COL 8.11

Office hours Bookable via Student Hub

Mondays 1-2

- ST443 teaches you:
  - mathematical ideas behind machine learning models
  - How to implement models using Python.

- Assessment: Exam (100%) 2 hours in the Spring exam period.

Attendance

- If you are unable to make it to your usual class, attend any other that week if possible, and let me know in class the following week.

- No need to email me about missing class unless you will miss 2 or more consecutive classes.

ST443/SEMINAR/01 ST443 Machine Learning and Data Mining

Type	Grp	Day	Start	Finish	AT Wks	WT Wks	ST Wks	Room	All Teachers
SEM	1	Thu	09:00	10:30	1, 2, 3, 4, 5, 7, 8, 9, 10, 11			CBG.1.03	CHAN, DR PHILIP

ST443/SEMINAR/02 ST443 Machine Learning and Data Mining

Type	Grp	Day	Start	Finish	AT Wks	WT Wks	ST Wks	Room	All Teachers
SEM	2	Fri	10:30	12:00	1, 2, 3, 4, 5, 7, 8, 9, 10, 11			CBG.1.06	CHAN, DR PHILIP

ST443/SEMINAR/03 ST443 Machine Learning and Data Mining

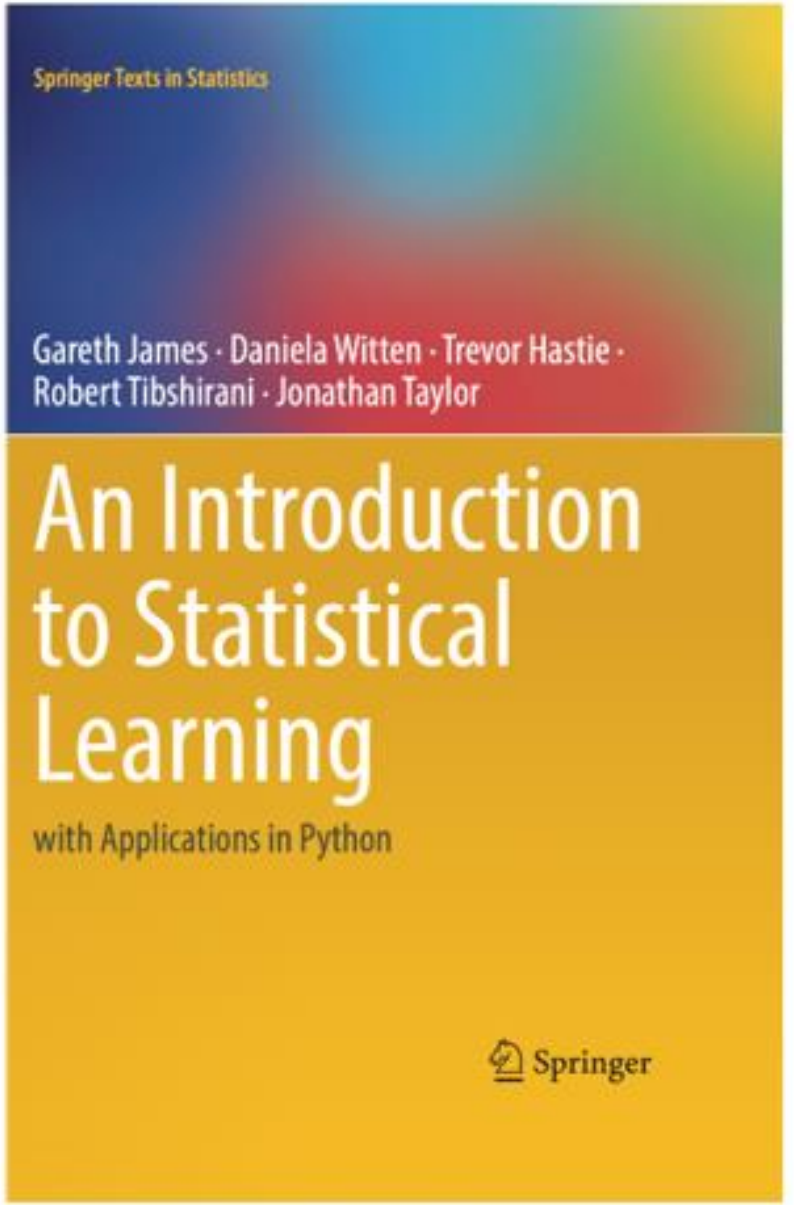
Type	Grp	Day	Start	Finish	AT Wks	WT Wks	ST Wks	Room	All Teachers
SEM	3	Thu	10:30	12:00	1, 2, 3, 4, 5, 7, 8, 9, 10, 11			CBG.1.03	CHAN, DR PHILIP

ST443/SEMINAR/04 ST443 Machine Learning and Data Mining

Type	Grp	Day	Start	Finish	AT Wks	WT Wks	ST Wks	Room	All Teachers
SEM	4	Fri	17:00	18:30	1, 2, 3, 4, 5, 7, 8, 9, 10, 11			CBG.2.06	ZHOU, DR KAIFANG

Seminars

- **Practical applications** with Python based on ISLP (7th ed)



An Introduction to Statistical Learning

Seminars

- ML concepts
- Implementation of ML models in Python
- Theory



Jupyter Notebook

- Jupyter Notebook is an open-source web application that allows you to create and share documents that contain
    - Code and output
    - Visualisations
    - Narrative text (including equations)

Jupyter Notebook: Use

- Data cleaning and transformation
- Data visualisation
- Numerical simulation
- Statistical modeling
- Machine learning

and many more.



Colab as a version of Jupyter that runs on Google's cloud infrastructure, providing access to computing resources without any local installation



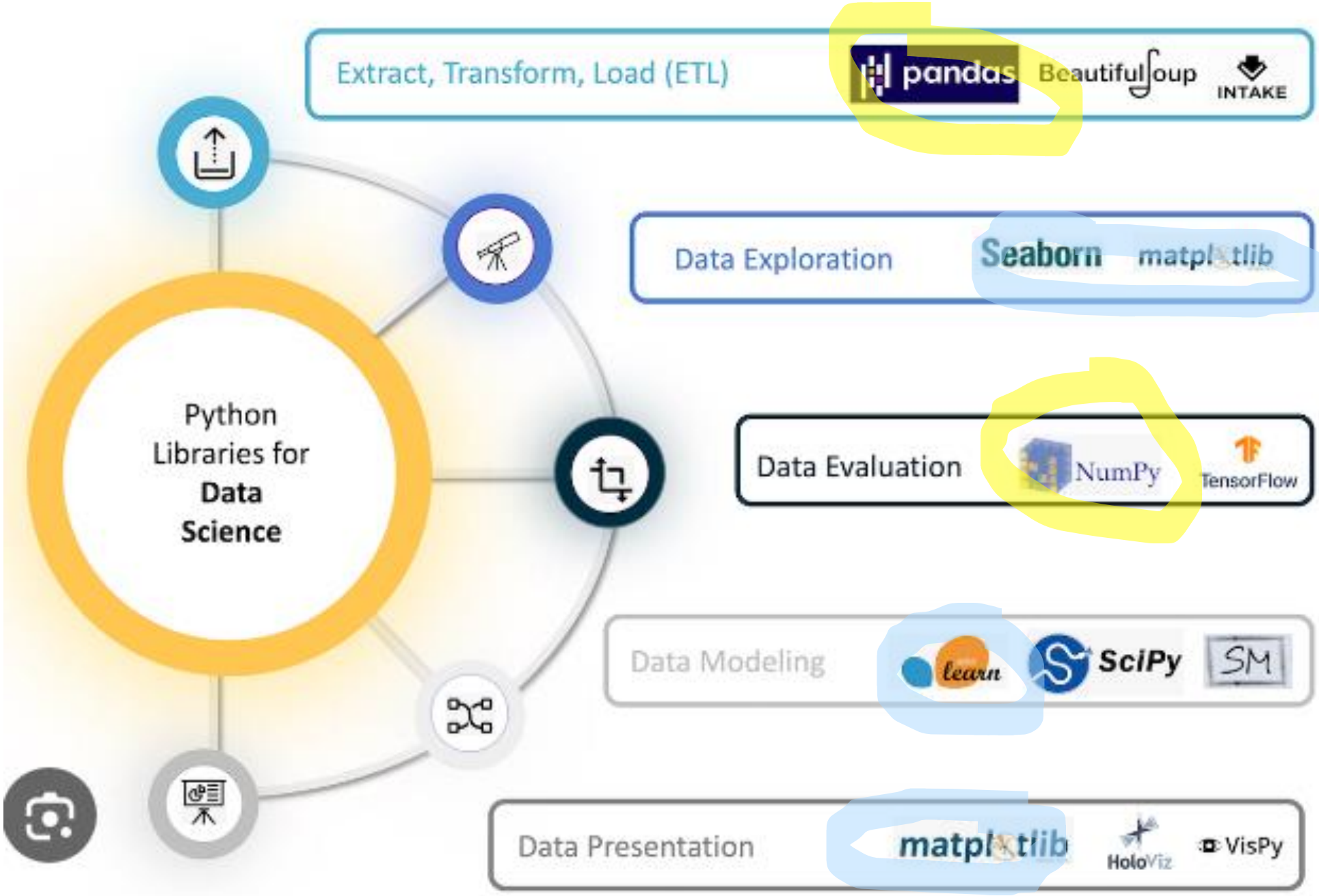
Notebook |Getting started

ST443 seminar slides



Python Libraries

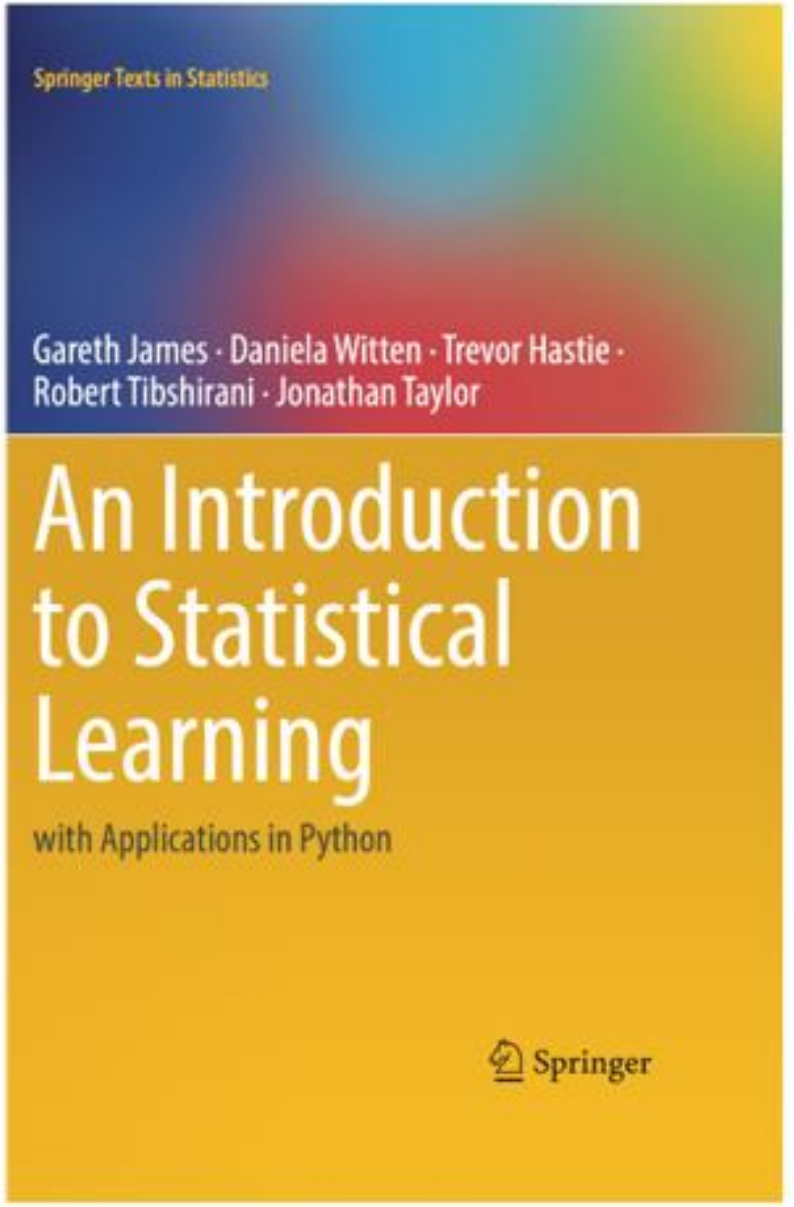



Extract, Transform, Load (ETL)
pandas
Beautiful Soup
INTAKE
Data Exploration
Seaborn
matplotlib
Python Libraries for Data Science
Data Evaluation
NumPy
TensorFlow
Data Modeling
learn
SciPy
SM
Data Presentation
matplotlib
HoloViz
VisPy

Notebooks

Link to all complete notebooks accompanying ISLP

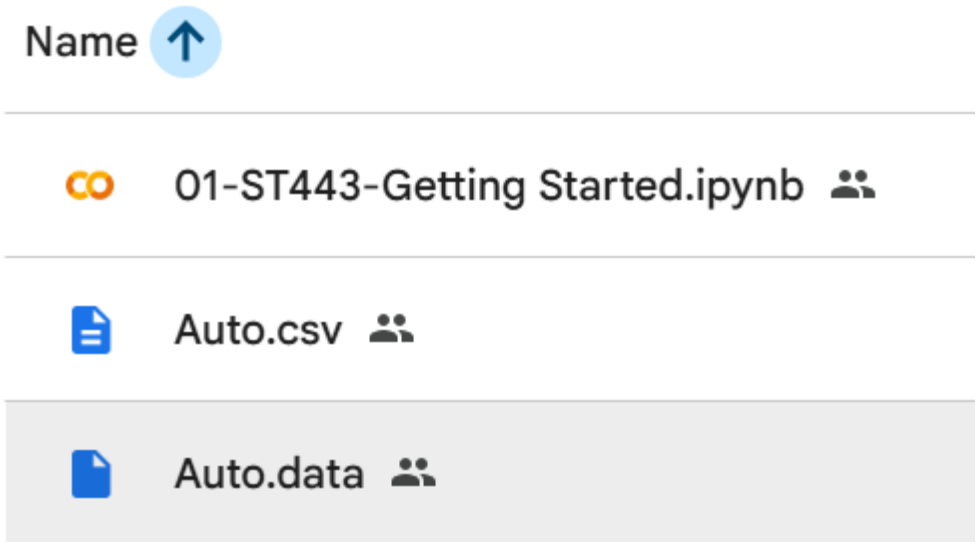
https://github.com/intro-stat-learning/ISLP_labs/tree/stable



Seminar 1 notebook

- Create a folder for this course, call it say, ST443
- Log in to your Google Drive account
- Save a copy of the files to your Drive
- Link to the file (or use QR)

https://drive.google.com/drive/folders/1P2XVUWGgMq5A9fb_HBk1oRMGcFMFGkKt?usp=sharing





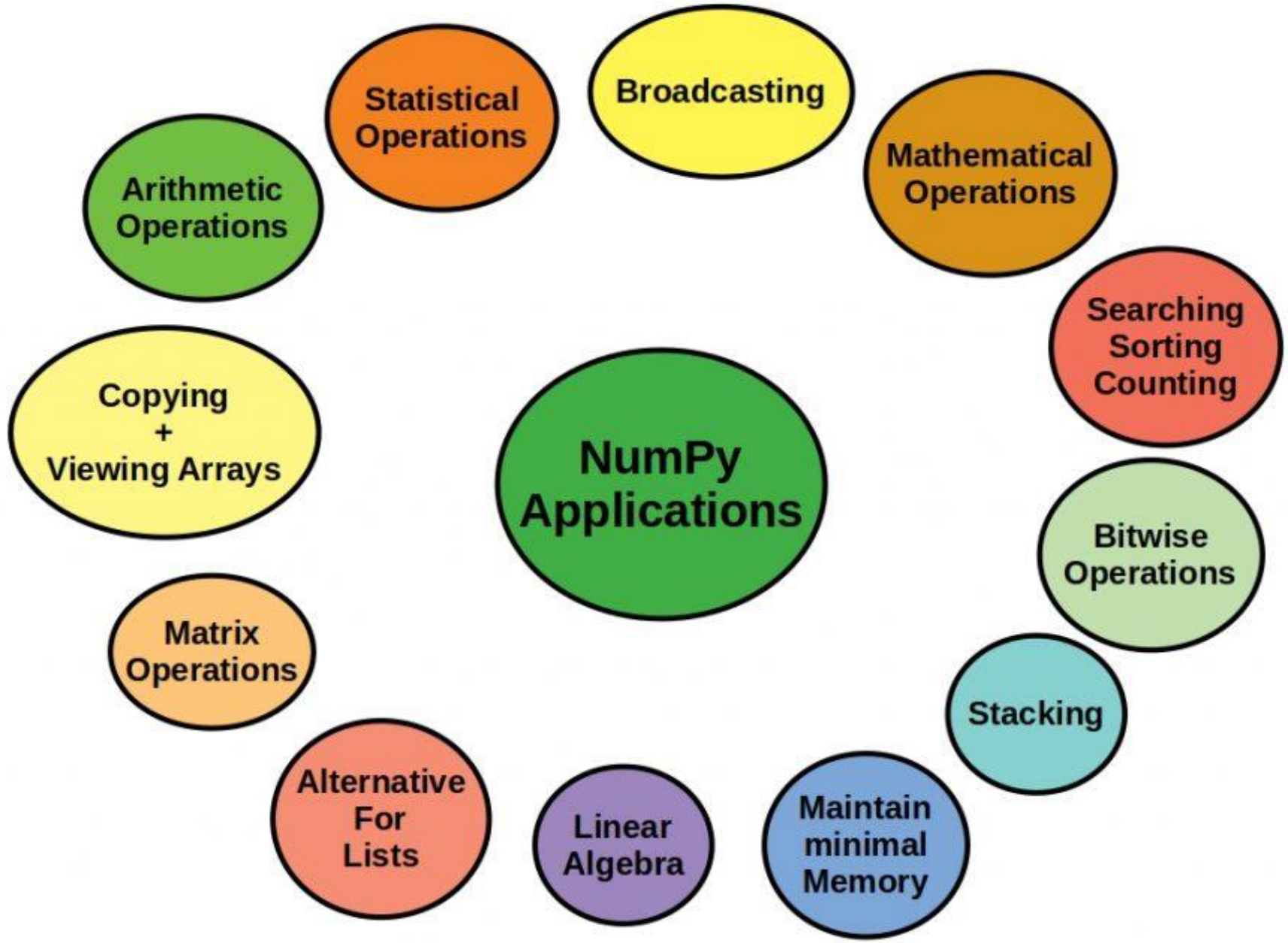
Notebook |NumPy

NumPy is an important library for:



- Data Science
- Machine learning
- Signal and image processing
- Scientific and engineer computing

https://numpy.org/devdocs/user/absolute_beginners.html


NumPy Applications
Statistical Operations
Broadcasting
Mathematical Operations
Arithmetic Operations
Searching Sorting Counting
Copying + Viewing Arrays
Bitwise Operations
Matrix Operations
Stacking
Alternative For Lists
Linear Algebra
Maintain minimal Memory



NumPy

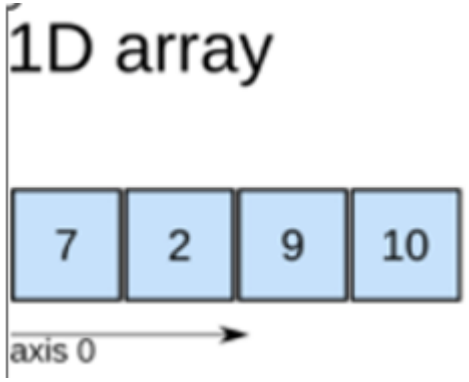
- NumPy = Numerical Python
- Is a Python library for performing fast and space-efficient numerical calculations on arrays.
- NumPy can replace use of loops using array expressions

https://www.oreilly.com/library/view/python-for-data/9781449323592/ch04.html

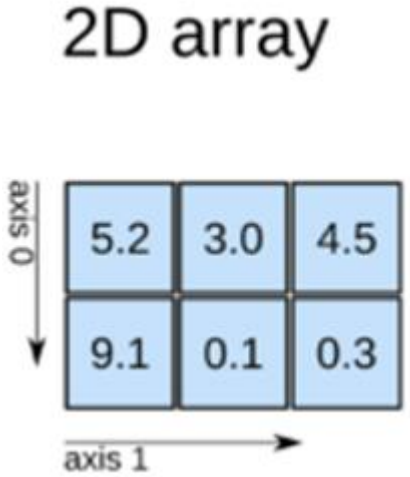
The key concept in NumPy is the **NumPy array**.

A NumPy array is a grid of values, all of the same type, and is indexed by a tuple of nonnegative integers.

A NumPy array may have one or more dimensions



A 1D array is a vector



A 2D array is a matrix

https://www.pythontutorial.net/python-numpy/what-is-numpy/

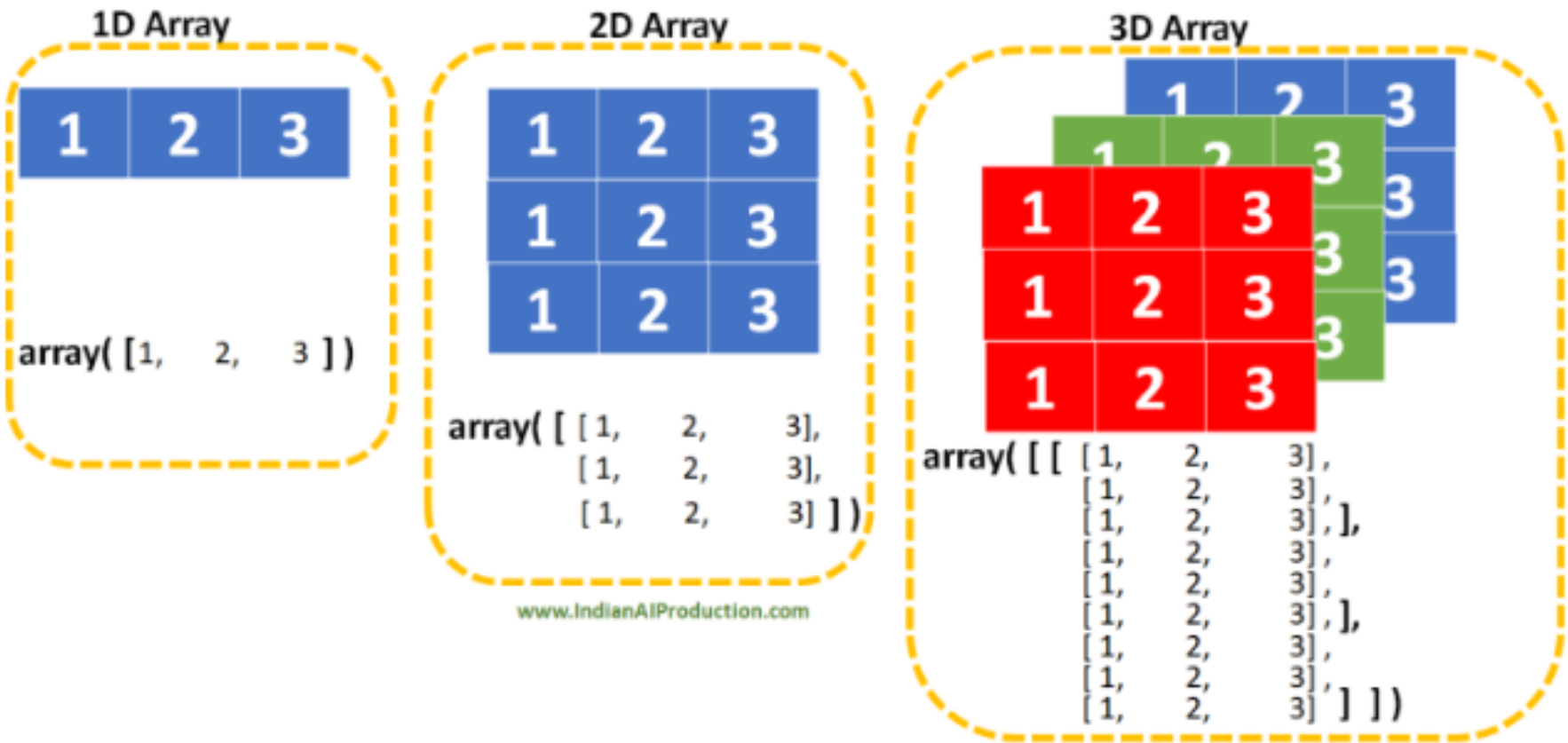


Fig 1.1 Representation of NumPy ndarray (multidimensional array)



What is the output of the following code?

```
import numpy as np

my_list = np.array([1,2],[3,4])
my_list
```

A. 1D array
B. 2D array
C. Something else

```
import numpy as np

my_list = np.array([1,2],[3,4])
my_list
```

```
  Cell In[3], line 1
    my_list = np.array([1,2],[3,4])

TypeError: Field elements must be 2- or 3-tuples, got '3'
```

The NumPy array function accepts any **ONE** sequence-like object (including other arrays) and produces a new NumPy array containing the passed data.



```
my_list = np.array([1,2],[3,4])
```

More than one object: 2 lists



```
my_list = np.array([[1,2],[3,4]])
```

One object: list of a list for 2D array

```
array([[1, 2],
       [3, 4]])
```

The array object in NumPy is called ndarray = N-dimensional array

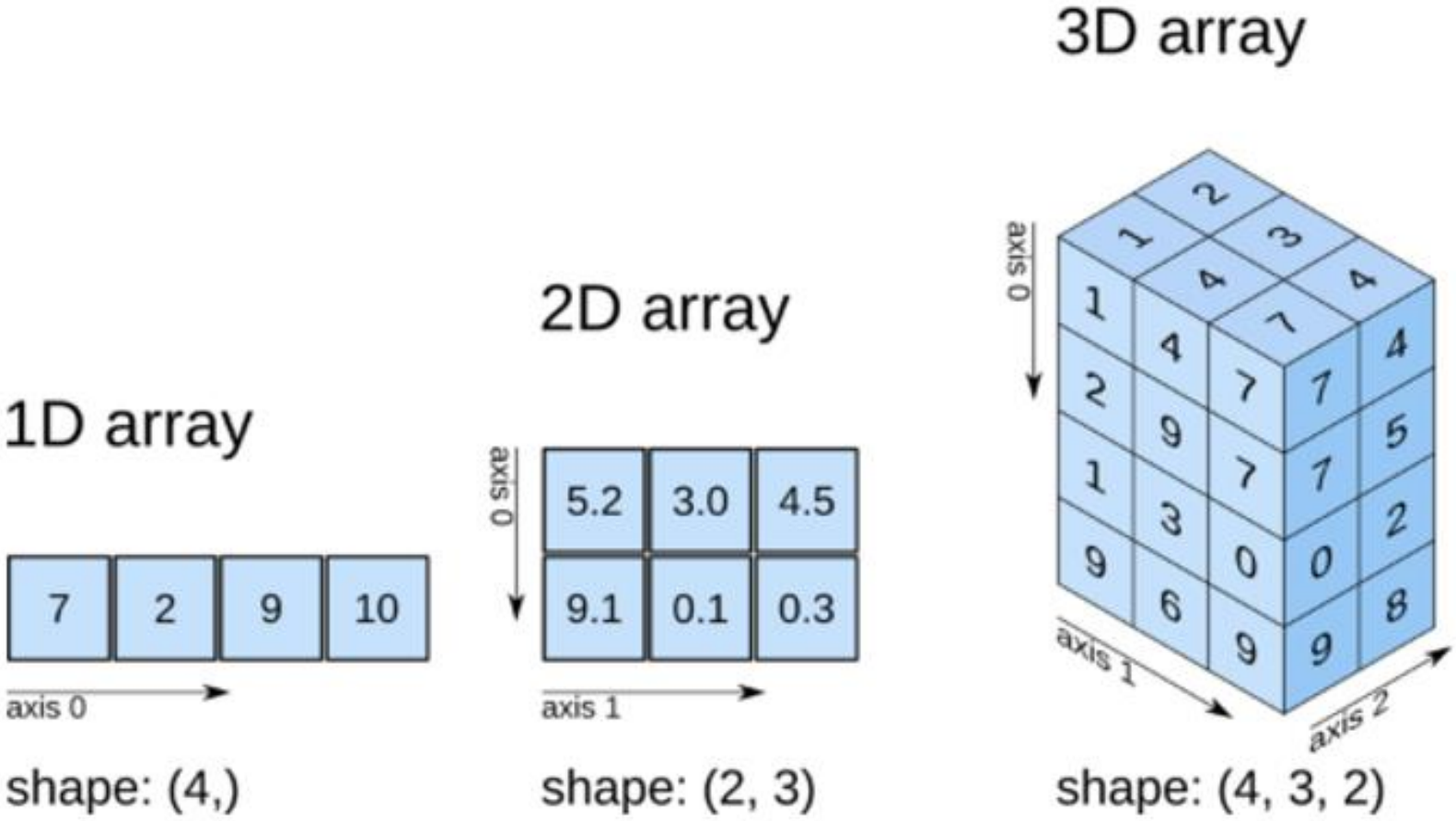
```
import numpy as np

A = np.array([[1,2,3],[4,5,6]])

type(A)
```

```
numpy.ndarray
```

- In NumPy dimensions are called *axes*.
- The number of elements in an axis is the length (of that axis)
- The shape of an array is a **tuple** consisting of the length in each axis



1 axis
of length 4

2 axes
Axis 0 has length 2
Axis 1 has length 3

3 axes
Axis 0 has length 4
Axis 1 has length 3
Axis 2 has length 2

https://numpy.org/devdocs/user/quickstart.html



What dimension is the array?
What is the shape of the array?

```
import numpy as np

A = np.array([[1,2,3],[4,5,6]])
```

Use *ndim* attribute to get the number of dimensions of a NumPy array.

```
In [10]: A = np.array([[1,2,3],[4,5,6]])
    ...: A.ndim
Out[10]: 2
```

NumPy arrays have an attribute called shape that returns
a tuple with each index having the number of corresponding elements.

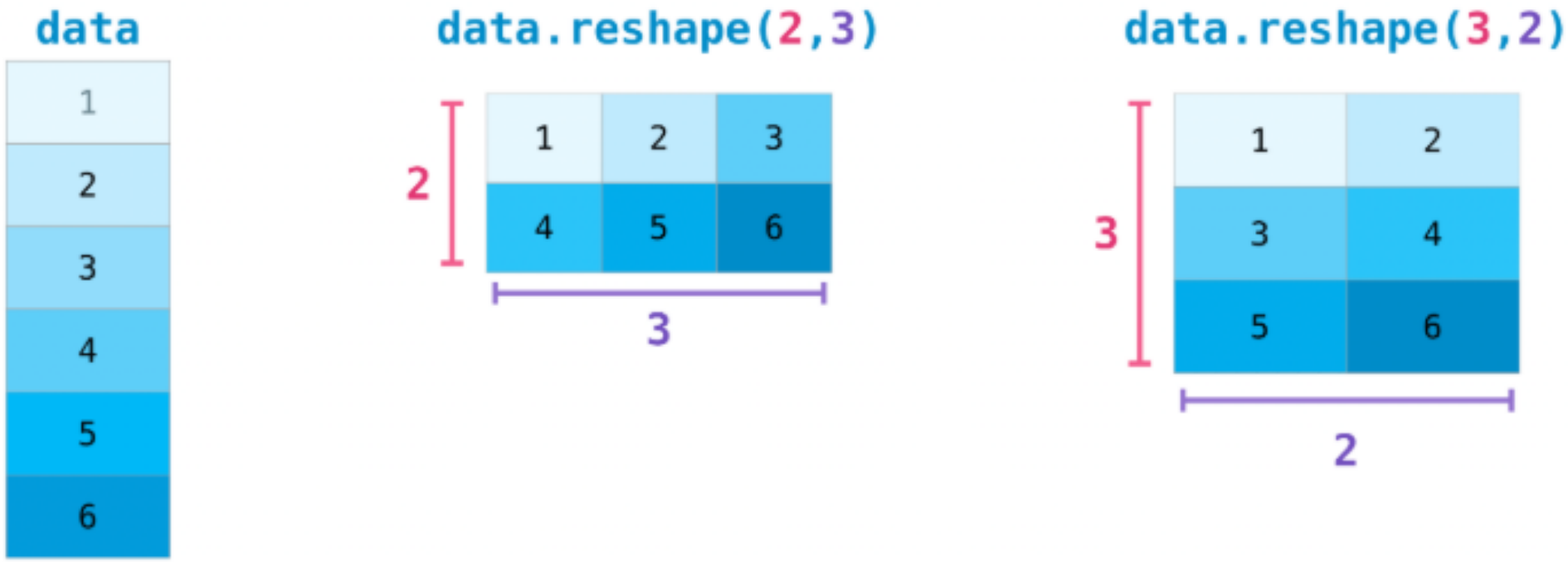
```
A = np.array([[1,2,3],[4,5,6]])
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

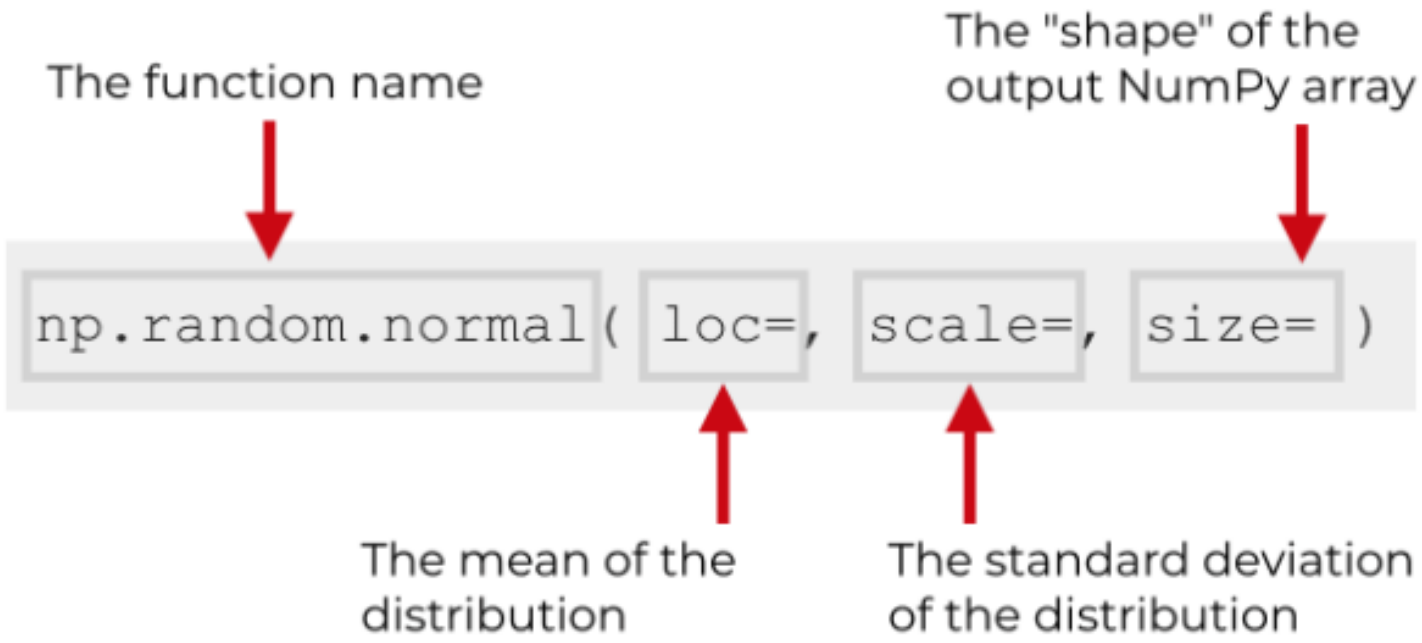
```
In [11]: A.shape
Out[11]: (2, 3)
```

https://www.w3schools.com/python/numpy/numpy_array_shape.asp

Reshaping an array



Generating random numbers



https://numpy.org/doc/2.1/reference/random/generated/numpy.random.normal.html



```
x = np.random.normal(size=5)
y = x + np.random.normal(loc=50, scale=1, size=5)
```

What is the value of :

- E(Y)
- Var(Y)

Get an estimate of these values using numpy

Random seed

```
rng = np.random.default_rng(1303)
print(rng.normal(scale=5, size=2))
rng2 = np.random.default_rng(1303)
print(rng2.normal(scale=5, size=2))
```

```
[ 4.09482632 -1.07485605]
[ 4.09482632 -1.07485605]
```

https://numpy.org/doc/2.2/reference/random/generator.html

Notebook |NumPy

Indexing and slicing (comes after Matplotlib)

Indexing in Python

Indexing is the process of accessing an element in a sequence using its position in the sequence (its index).

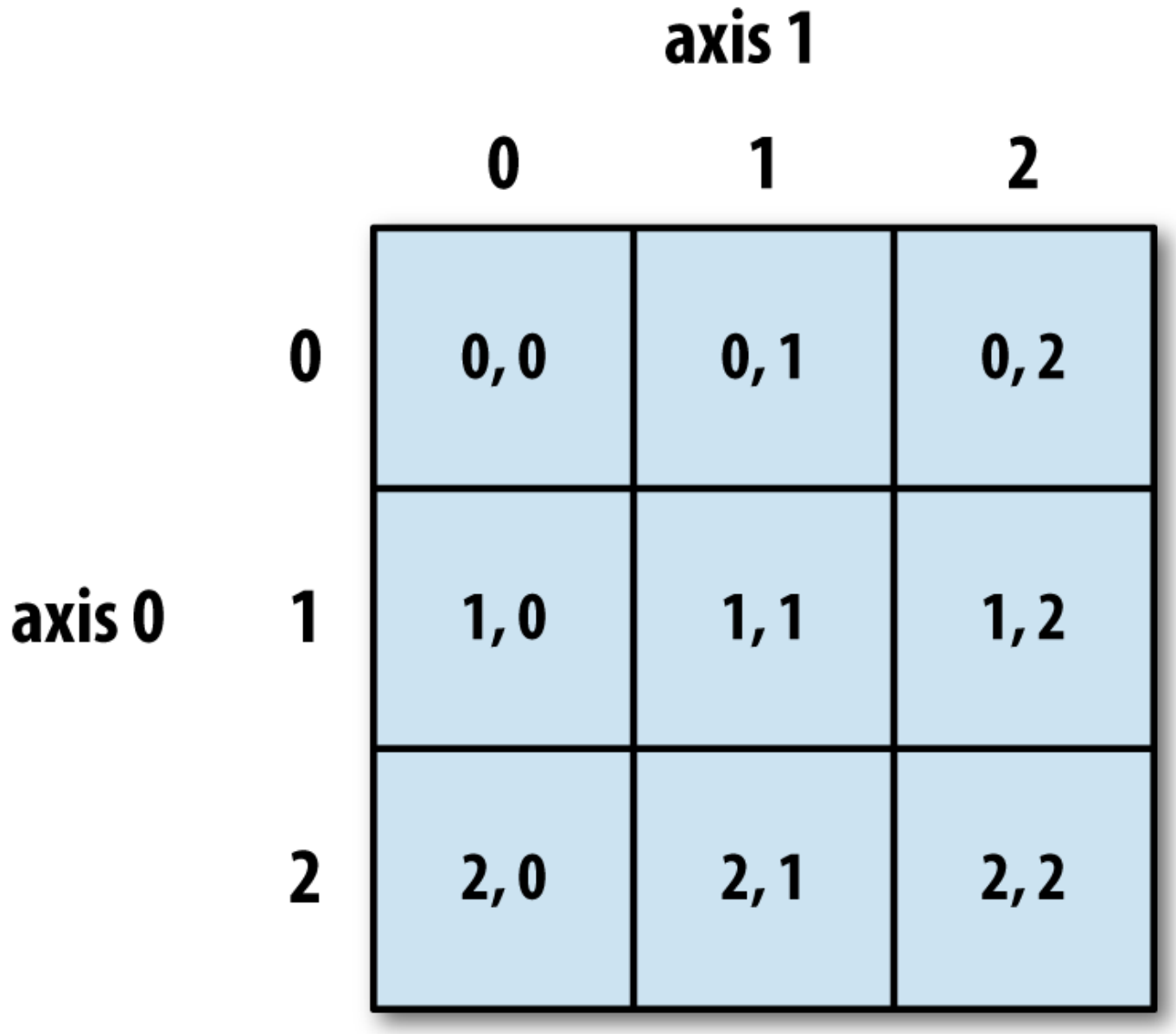
Indexing works on tuples and lists as they are sequences.

To access an element in a sequence, you can use square brackets `[]` with the index of the element you want to access.

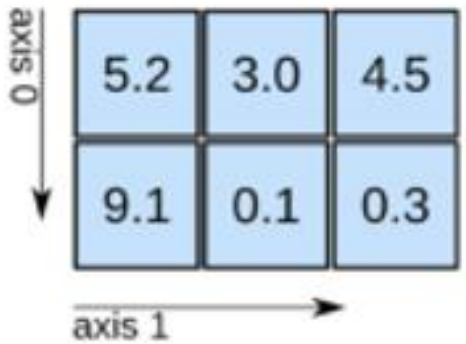
```
my_list = ['apple', 'banana', 'cherry', 'date']
print(my_list[0]) # output: 'apple'
print(my_list[1]) # output: 'banana'
```

https://www.freecodecamp.org/news/slicing-and-indexing-in-python/

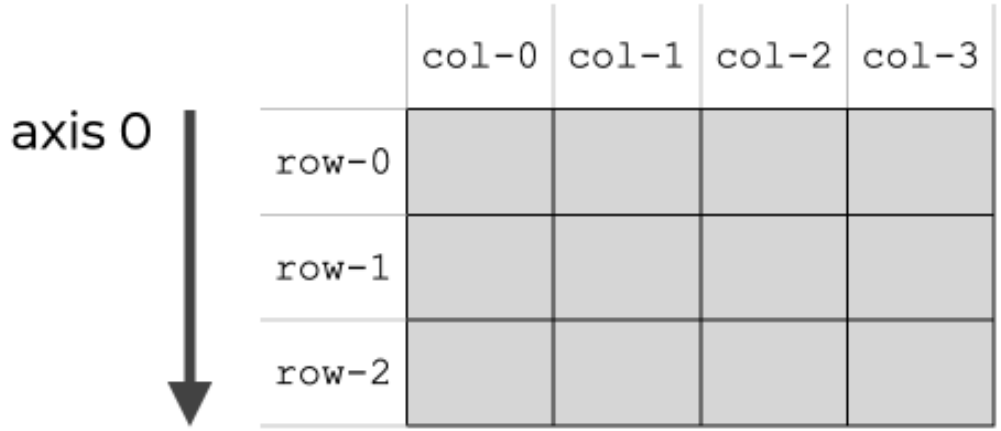
Indexing elements in a NumPy 2D array

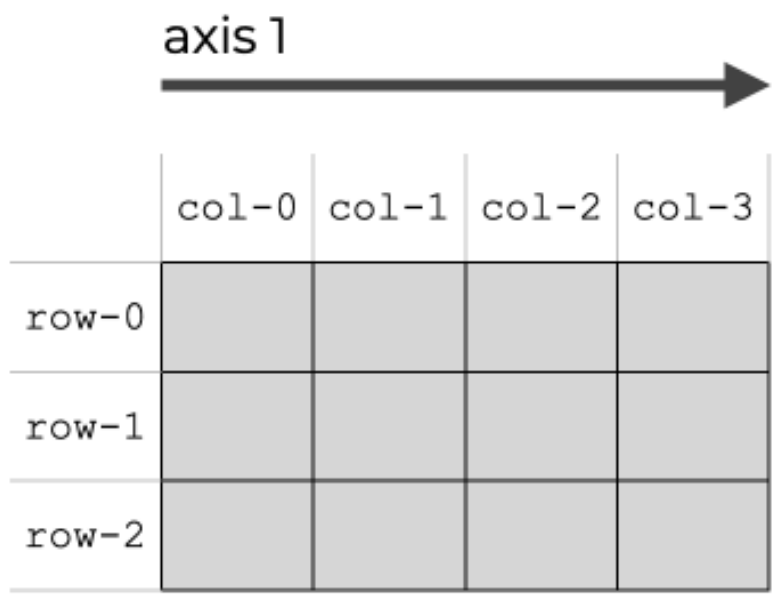


2D array



shape: (2, 3)





Slicing in Python

Slicing is the process of accessing a sub-sequence of a sequence by specifying a starting and ending index. In Python, you perform slicing using the colon `:` operator. The syntax for slicing is as follows:

```
sequence[start_index:end_index]
```

```
my_list = ['apple', 'banana', 'cherry', 'date']
print(my_list[1:3]) # output: ['banana', 'cherry']
```

Slicing 2-D Arrays

2D NumPy arrays can be sliced with the general form:

```
<slice> = <array>[start_row:end_row, start_col:end_col]
```

```
import numpy as np



a = np.array([[2, 4, 6, 8], [10, 20, 30, 40]])
print(a)
```

```
[[ 2  4  6  8]
 [10 20 30 40]]
```

```
a = np.array([[2, 4, 6, 8], [10, 20, 30, 40]])
```

```
b = a[0:2, 0:3]
print(b)
```

Output

```
[[ 2  4  6]
 [10 20 30]]
```

```
<slice> = <array>[start_row:end_row, start_col:end_col]
```



What is displayed?

```
A = np.arange(1, 17).reshape(4, 4)
print(A)
```

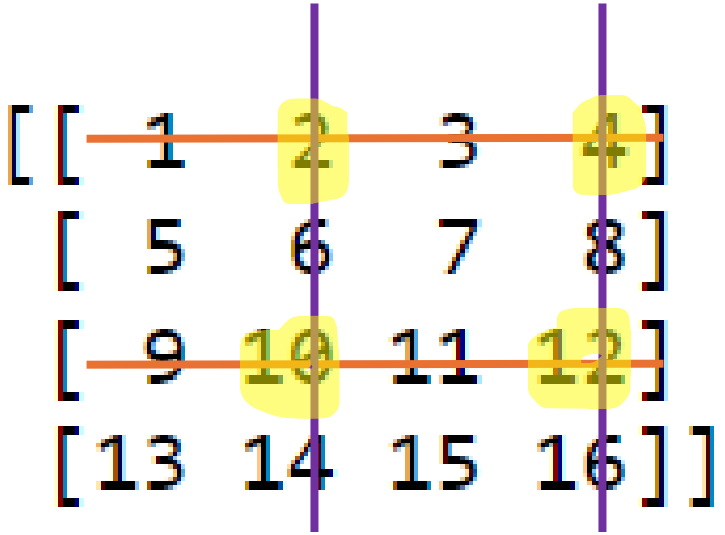
```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
```

```
print(A[[0, 2], :][:, [1, 3]])
```

This is an example of a subset of a subset. The first subset of A is given b `A[[0, 2], :]`

On this subset we extract fron it `[:, [1, 3]]`

```
A = np.arange(1, 17).reshape(4, 4)
print(A)
```



1st subset: select rows with index 0 and 2 and includes all columns

```
print(A[[0, 2], :][:, [1, 3]])
```

```
[[ 2  4]
 [10 12]]
```

From the 1st subset, select columns with index 1 and 3 and includes all rows

Slicing 2-D Arrays

2D NumPy arrays can be sliced with the general form:

```
<slice> = <array>[start_row:end_row, start_col:end_col]
```

```
import numpy as np



a = np.array([[2, 4, 6, 8], [10, 20, 30, 40]])
print(a)
```

```
[[ 2  4  6  8]
 [10 20 30 40]]
```



What is displayed by print(b)?

```
a = np.array([[2, 4, 6, 8], [10, 20, 30, 40]])
```

```
b = a[0:2, 0:3]
print(b)
```

```
a = np.array([[2, 4, 6, 8], [10, 20, 30, 40]])
```

```
b = a[0:2, 0:3]
print(b)
```

```
[[ 2  4  6]
 [10 20 30]]
```



Notebook |Matplotlib

https://matplotlib.org/



Notebook |pandas

What is pandas?



pandas is a data manipulation package:

- data cleaning
- data analysis

It is designed for working with 2D tabular data (think Excel).

https://www.datacamp.com/tutorial/pandas

pandas is used in tandem with :

- numerical computing tools e.g. NumPy,
- analytical libraries

1. Import pandas and load the data

It is very easy to load the CSV data using pandas. You can do so by using the function `pd.read_csv()` to load data to `pd.DataFrame`.

```
import pandas as pd

df = pd.read_csv('../data/coronavirus_data.csv', index_col=0, parse_dates=True)
```

That's it! How did we do it before?

Note:

- `index_col=0` means we want the column with index 0 to be the index
- `parse_dates=True` means to treat the index as datetime
- `read_csv()` has lots of optional arguments, see here for more details

The **index** of a DataFrame is a series of labels that identify each row. The labels can be integers, strings, or any other hashable type.

2. Inspect the data

Please always have a look and make sure your data is loaded properly. You can read the first few rows by `.head()` `.tail()`.

```
df.tail()
```

Date_reported	Canada	France	Germany	Italy	Japan	The United Kingdom	United States of America
2021-10-04	0	3744	3088	2967	963	29666	0
2021-10-05	0	402	4799	1612	732	34256	168684
2021-10-06	11464	4611	11547	2463	897	33049	70915

The index, which identifies the row

3. Calculate some specific statistic

With `pandas`, it is very easy to calculate some statistics, for example:

[Q3b. Calculate the total number of cases for all 7 countries for each day]

We can do so by using the method `sum()`:

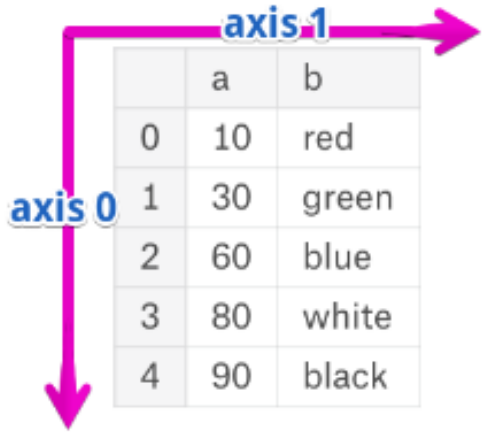
```
df.sum(axis=1)
```

Cases across all countries for that date

```
Date_reported
2020-01-03        0
2020-01-04        0
2020-01-05        0
2020-01-06        0
2020-01-07        0
                 ...
2021-10-04    40428
2021-10-05   210485
2021-10-06   134946
2021-10-07   172336
2021-10-08   175301
Length: 645, dtype: int64
```

A DataFrame object has two axes: “axis 0” and “axis 1”.

- axis 0 represents *across* rows
- axis 1 represents *across* columns.



How do I select specific columns from a `DataFrame`?



Each column in a DataFrame is a **Series**

To select a single column, use square brackets [] with the column name of the column of interest.

4. Extracting a specific column

[Q3d. Extract the column for Japan]

This is extremely easy with `pandas` :

```
df['Japan']                                    # DataFrame[‘<name of column>’]
```

```
Date_reported
2020-01-03       0
2020-01-04       0
2020-01-05       0
2020-01-06       0
2020-01-07       0
                ...
2021-10-04     963
2021-10-05     732
```

Date_reported	Canada	France	Germany	Italy	Japan
2021-10-04	0	3744	3088	2967	963
2021-10-05	0	402	4799	1612	732
2021-10-06	11464	4611	11547	2463	897

DataFrame called df

[Q3c. Calculate the total number of cases for each country in the first 6 months in 2021]

We can do so by using boolean indexing. We first get the year and `month` information using `.year` and `.month`, just like how we did with `datetime`, as the index is `DatetimeIndex`:

In [6]:
```
df.index
```

Out[6]:
```
DatetimeIndex(['2020-01-03', '2020-01-04', '2020-01-05', '2020-01-06',
               '2020-01-07', '2020-01-08', '2020-01-09', '2020-01-10',
               '2020-01-11', '2020-01-12',
               ...
               '2021-09-29', '2021-09-30', '2021-10-01', '2021-10-02',
               '2021-10-03', '2021-10-04', '2021-10-05', '2021-10-06',
               '2021-10-07', '2021-10-08'],
              dtype='datetime64[ns]', name='Date_reported', length=645, freq=None)
```

```
df.index.year                    # Output variable year of the DatetimeIndex()
```

```
Int64Index([2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020, 2020,
            ...
            2021, 2021, 2021, 2021, 2021, 2021, 2021, 2021, 2021, 2021],
           dtype='int64', name='Date_reported', length=645)
```

A DatetimeIndex()

https://pandas.pydata.org/docs/reference/api/pandas.DatetimeIndex.html

Extract the rows that are in the 1st 6 months of 2021, and save it as a data frame

```
df_2021_first_half = df[(df.index.year == 2021) & (df.index.month <= 6)]
```

Sum across rows of the data frame

```
df_2021_first_half.sum() # default is axis=0
```

```
Canada                          848628
France                         3097201
Germany                        2008404
Italy                          2175444
Japan                           567855
The United Kingdom             2242704
United States of America      13804472
dtype: int64
```

Date_reported	Canada	France	Germany	Italy	Japan
2021-10-04	0	3744	3088	2967	963
2021-10-05	0	402	4799	1612	732
2021-10-06	11464	4611	11547	2463	897

